Python boolean data types;

→ it represents the two logical states

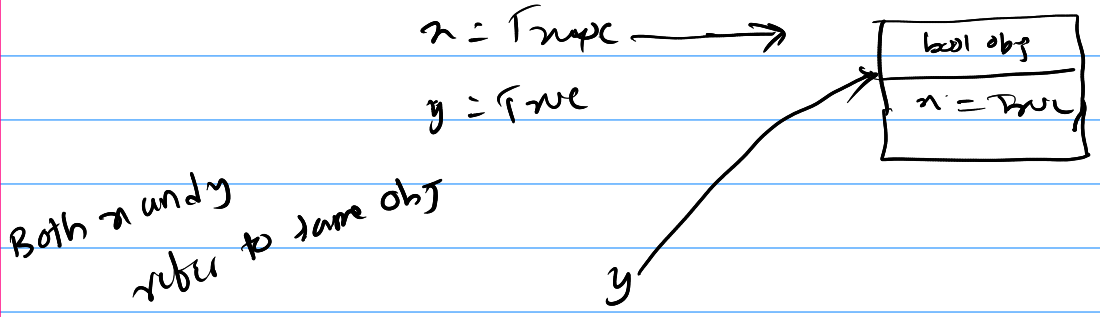
① True
② False

To find it type:-

```
x = True
print(type(x))        # <class bool>
```

What happens in memory:-

→ There only two values True/false. if we create more variables for True/False. Then those variables refer to first created object



→ here if you find the(id) of the both variable both are same.

boolean operators :

① And

① and

True AND True → True
True AND False → False
False AND True → False
False AND False → False

② OR

True and True → True
True and False → True
false and True → True
false and false → False

• Not

NOT True
→ False
NOT False
→ True.

Truthiness :

→ python doesn't requires an actual True or false in every if.

Eg :-

```
x = 10
if x :          ← This works Bcz x → Truthy
    print("Hello")                    false
```

common false values :

→ False, None, 0, 0.0, "", [], (), {}, set()

Binary :-

→ Binary data means represents sequence of bytes where each byte contains a value from 0 to 255

→ 1 byte = 8 bits

1 byte = 8 bits

→ for example H → 72

↓

01001000

8 4 2 1

1 6
1 6
3 2

→ so, when python work with raw data such as images, pdf, audio, videos, n/w packets, encrypted data, it can be represented that data using binary data types

Binary data types :-

Bytes :-

→ it represents sequence of bytes in python

→ and it is immutable sequence of integers from 0-255

```
d = b"Hello"
print(d)
```

Byte array :-

→ Bytearray is every similar to bytes. but it is mutable.

```
data = bytearray(b"Hello")
print(data)
data[0] = 72  ⟹ b"Jello"
```

byte vs bytearray

bytes	bytearray
↓	↓
read only binary data	Modified binary data.

→ memory view :-

-> memory view is not really another copy of binary data
-> it provide a way to view the memory object.

Ex :-

```
data = bytearray(b"Hello")
view = memoryview(data)
```

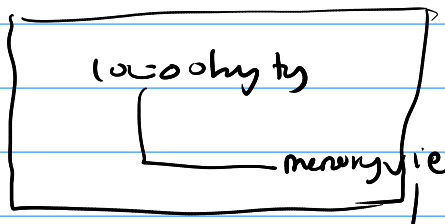
-> it basically says Give me access to existing memory

Let say :

we 100MB

-> we can use the only part of it

```
data = bytearray(100_000_000)
view = memoryview(data)
part = view[1000:2000]
```



This uses existing memory rather than create new one

Same memory

→ it can be useful when working with:

1. large files
2. N/w packets.
3. images
4. Video
5. binary protocols
6. Numpy array
7. Low-level data processing.

*None :-

→ None is a specibial python objut represent "no value" or "nothing"

x = None

→ x is a vaiible

→ None is a altual objut

→ x refers to None objut

→ it dos not mean 0, false, empty string, absence of value.

080-6234 6677

→ Why do we need None ?:-

→ Let say we are return a function with no value. so at the time we are using the None

example:-

```
def get_user():
    return None

user = get_user()
print(user) # None
```

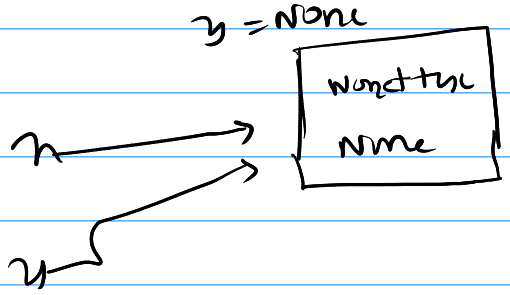
None with functions:-

→ if python function does not explicitly return anything, then it automatically return the None

```
def test():
    print("Hellow")
var = test()
print(var)   # None
```

Memory Lucd :-

```
x = None
y = None
```



Same obj id with TWO references

None-false:-

→ python considers None as falsy like condition

```
a = None
if a:
    print("me")
else:
    print("you")      # you
```

→ The None is connected onto a NULL in table(SQL)

```
{ a = "something"
  b = "something"
  c = None
```

⟹

a	b	c
something	something	NULL